

Documentation du Projet de Prédiction de l'Indice Boursier MASI 20

1.Introduction et Contexte Général :

La fluctuation des indices boursiers, tels que le MASI 20 (Moroccan All Shares Index), est une caractéristique inhérente aux marchés financiers. Ces mouvements, imprévisibles par nature, créent une incertitude significative pour les investisseurs. Pour les acteurs du marché, qu'ils soient institutionnels ou individuels, anticiper les tendances futures de l'indice peut offrir un avantage compétitif crucial, permettant d'optimiser les stratégies d'investissement, de minimiser les risques et de maximiser les rendements. La volatilité du MASI 20, influencée par des facteurs économiques nationaux et internationaux, des événements géopolitiques et le sentiment des investisseurs, rend cette tâche complexe mais essentielle. Ce projet vise à explorer la capacité des modèles de Machine Learning à prédire les mouvements de l'indice MASI 20, afin de fournir des outils d'aide à la décision face à cette incertitude.

2.Chargement des Données :

[Masi 20 Historical Data \(MSI20\) - Investing.com](#)

Objectif : Charger le fichier de données historiques du MASI 20 dans un format exploitable pour l'analyse.

```
import pandas as pd

df = pd.read_csv('/content/Masi 20 Historical Data.csv')
print(df.head())
```

	Date	Price	Open	High	Low	Vol.	Change %
0	12/16/2025	1,503.32	1,502.27	1,513.96	1,501.87	NaN	0.07%
1	12/15/2025	1,502.27	1,502.55	1,513.92	1,498.11	NaN	-0.02%
2	12/12/2025	1,502.55	1,502.40	1,511.85	1,500.61	NaN	0.01%
3	12/11/2025	1,502.40	1,504.71	1,506.76	1,495.54	NaN	-0.15%
4	12/10/2025	1,504.71	1,500.45	1,507.38	1,493.86	NaN	0.28%

Explication : La première étape cruciale a consisté à importer la bibliothèque pandas et à charger le fichier CSV nommé /content/Masi 20 Historical Data.csv.crdownload dans un DataFrame. Cette action garantit un point de départ propre pour toute manipulation ou analyse ultérieure des données, évitant les interférences d'exécutions précédentes. Les premières lignes du DataFrame ont été affichées pour confirmer le chargement correct et un aperçu initial des colonnes.

3. Inspection Initiale des Données :

Objectif : Comprendre la structure du DataFrame, les types de données, identifier les valeurs manquantes et obtenir des statistiques descriptives de base.

```
print("DataFrame Info:")
df.info()

print("\nDescriptive Statistics:")
print(df.describe(include='all'))

print("\nMissing Values:")
print(df.isnull().sum())
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 721 entries, 0 to 720
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        721 non-null    object
1   Price       721 non-null    object
2   Open        721 non-null    object
3   High        721 non-null    object
4   Low         721 non-null    object
5   Vol.        0 non-null      float64
6   Change %    721 non-null    object
dtypes: float64(1), object(6)
```

Explication : Cette phase exploratoire a permis de dresser un portrait initial du jeu de données. Des méthodes comme `df.info()` ont révélé les types de données de chaque colonne et la présence de valeurs non nulles.

`df.describe(include='all')` a fourni des statistiques sommaires pour toutes les colonnes, numériques et catégorielles, tandis que `df.isnull().sum()` a mis en évidence le nombre de valeurs manquantes par colonne. Cette inspection a confirmé que les colonnes financières étaient initialement de type `object` (chaîne de caractères), nécessitant un nettoyage, et que la colonne 'Vol.' (Volume) était entièrement vide.

4. Pré-traitement des Données (Approche Unidimensionnelle):

Objectif : Appliquer les étapes de nettoyage et de transformation nécessaires, en se concentrant uniquement sur les colonnes 'Date' et 'Price' pour une prédiction unidimensionnelle.

```
import pandas as pd
import numpy as np

# Reload the DataFrame to ensure 'Price' column is a string type for .str access
# This is necessary because previous execution already converted 'Price' to float
df = pd.read_csv('/content/Masi 20 Historical Data.csv.crdownload')

# 1. Remove commas from 'Price' and convert to numeric (float)
df['Price'] = df['Price'].str.replace(',', '', regex=False).astype(float)

# 2. Convert 'Date' to datetime objects
df['Date'] = pd.to_datetime(df['Date'])

# 3. Set 'Date' as the DataFrame index
df = df.set_index('Date')

# 4. Drop all columns except for 'Price'
df = df[['Price']]

# 5. Check for and handle any remaining missing values in the 'Price' column
# Fixed: Use .ffill() directly and assign back to avoid FutureWarning.
```

Explication : Conformément à l'objectif de prédiction unidimensionnelle, le pré-traitement s'est focalisé sur les deux colonnes essentielles : 'Date' et 'Price'.

Nettoyage de la colonne 'Price' : Les virgules ont été retirées des valeurs de 'Price', qui ont ensuite été converties en nombres décimaux (float).

Conversion de la colonne 'Date' : La colonne 'Date' a été convertie en objets `datetime` pour faciliter les opérations temporelles.

Indexation par date : La colonne 'Date' a été définie comme index du DataFrame, ce qui est standard pour les séries temporelles et permet un tri chronologique implicite.

Sélection des colonnes : Toutes les colonnes, à l'exception de 'Price', ont été supprimées du DataFrame, laissant un ensemble de données unidimensionnel.

Gestion des valeurs manquantes : Les valeurs manquantes éventuelles dans la colonne 'Price' ont été remplies par la méthode de propagation vers l'avant (`ffill()`), une approche courante en séries temporelles pour maintenir la continuité des données.

Cette étape a abouti à un DataFrame `df` propre, trié chronologiquement et contenant uniquement la colonne 'Price' comme série temporelle à prédire.

4. Préparation des Données pour l'Entraînement (Unidimensionnelle):

Objectif : Transformer la série de prix unidimensionnelle en un format adapté à l'entraînement d'un modèle LSTM, incluant la création de

séquences, la mise à l'échelle et la division en ensembles d'entraînement et de test.

https://colab.research.google.com/drive/1Ndws0beuccxbGpM6aBbfuRcPb-Zbb_0X#scrollTo=8z_RhZ_EbBU3&line=41&uniqifier=1

Explication : Cette étape a préparé la série de prix pour les modèles de prédiction de séries temporelles.

Tri chronologique : Le DataFrame a été trié par date en ordre croissant pour assurer la cohérence temporelle.

Extraction des données : Les valeurs de la colonne 'Price' ont été extraites dans un tableau NumPy, puis remodelées en un format (n_échantillons, 1) pour être compatibles avec le MinMaxScaler.

Mise à l'échelle des données : Un MinMaxScaler a été appliqué pour normaliser les prix entre 0 et 1. Cette mise à l'échelle est cruciale pour les réseaux neuronaux, car elle accélère la convergence et améliore la stabilité de l'entraînement.

Création de séquences (look-back) : Une fonction create_dataset a été utilisée pour créer des paires d'entrée-sortie. Pour chaque point de données, l'entrée (X) est le prix du jour précédent (look_back=1), et la sortie (y) est le prix du jour actuel. Cela transforme le problème de prédiction en un problème d'apprentissage supervisé.

Division en ensembles d'entraînement et de test : Les données ont été divisées en ensembles d'entraînement (80 %) et de test (20 %) en respectant l'ordre temporel strict. X_train, y_train représentent l'historique pour l'apprentissage, tandis que X_test, y_test sont utilisés pour évaluer la performance du modèle sur des données non vues.

Remodelage pour LSTM : Les ensembles X_train et X_test ont été remodelés en un format 3D (nombre_d'échantillons, look_back, nombre_de_caractéristiques), soit (574, 1, 1) pour l'entraînement et (143, 1, 1) pour le test. Ce format est le requis pour les modèles LSTM.

5. Entraînement et Évaluation des Modèles

Deux modèles de régression ont été entraînés et évalués pour prédire le MASI 20 :

5.1. Modèle de Régression Linéaire:

Objectif : Entraîner un modèle de régression linéaire, faire des prédictions et évaluer sa performance.

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
import matplotlib.pyplot as plt

# The original 'data' variable is already available from previous steps and is an array of original prices.
# We need a scaler to inverse transform the predictions and actual values.
# Create an instance of MinMaxScaler and fit it to the original 'data'.
scaler_inverse = MinMaxScaler(feature_range=(0, 1))
scaler_inverse.fit(data)

# Reshape X_train_resaped and X_test_resaped from 3D to 2D for Linear Regression
X_train_lr = X_train_resaped.reshape(X_train_resaped.shape[0], X_train_resaped.shape[1] * X_train_resaped.shape[2])
X_test_lr = X_test_resaped.reshape(X_test_resaped.shape[0], X_test_resaped.shape[1] * X_test_resaped.shape[2])

# FIX: Ensure y_train has the same number of samples as X_train_lr
# The error message indicates X_train_lr has 574 samples and y_train has 576 samples.
# Truncate y_train to match the length of X_train_lr.
if len(y_train) != X_train_lr.shape[0]:
    print(f"Warning: y_train length ({len(y_train)}) inconsistent with X_train_lr length ({X_train_lr.shape[0]}). Truncating y_train to match X_train_lr length.")
```

Explication :

- 1. Préparation des données :** Les données d'entrée X_train_resaped et X_test_resaped ont été remodelées de 3D à 2D (X_train_lr, X_test_lr) car la régression linéaire attend un format 2D. Des ajustements ont été faits pour s'assurer que y_train

et `y_test` avaient le même nombre d'échantillons que leurs homologues `X` respectifs.

2. **Entraînement** : Un modèle `LinearRegression` de `sklearn` a été initialisé et entraîné sur `X_train_lr` et `y_train`.
3. **Prédiction** : Le modèle a ensuite prédit les prix sur `X_test_lr`.
4. **Transformation inverse** : Les prédictions et les valeurs réelles du test ont été transformées inversement à leur échelle de prix originale à l'aide du `scaler_inverse`.
5. **Évaluation** : Les métriques suivantes ont été calculées :
 - a. **MSE (Mean Squared Error) : 237.3490**
 - b. **RMSE (Root Mean Squared Error) : 15.4061**
 - c. **MAE (Mean Absolute Error) : 11.1892**
6. **Visualisation** : Un graphique a comparé les prix réels et les prix prédits par le modèle de régression linéaire sur l'ensemble de test, montrant une bonne adéquation.

5.2. Modèle de Régression Forêt Aléatoire (Random Forest)

Objectif : Entraîner un modèle de régression Forêt Aléatoire, faire des prédictions et évaluer sa performance.

https://colab.research.google.com/drive/1Ndws0beuccxbGpM6aBbfuRcPb-Zbb_0X#scrollTo=9cc2e924&line=16&uniqifier=1

Explication :

1. **Entraînement** : Un modèle `RandomForestRegressor` de `sklearn` (avec `n_estimators=100`, `random_state=42`) a été initialisé et entraîné sur `X_train_lr` et `y_train`.

2. **Prédiction** : Le modèle a prédit les prix sur `X_test_lr`.
3. **Transformation inverse** : Les prédictions ont été transformées inversement à leur échelle de prix originale.
4. **Évaluation** : Les métriques suivantes ont été calculées :
 - a. **MSE (Mean Squared Error) : 10038.0784**
 - b. **RMSE (Root Mean Squared Error) : 100.1902**
 - c. **MAE (Mean Absolute Error) : 83.4546**
5. **Visualisation** : Un graphique a comparé les prix réels et les prix prédits par le modèle de forêt aléatoire sur l'ensemble de test, montrant une moins bonne adéquation que la régression linéaire.

6. Comparaison des Performances des Modèles

Objectif : Déterminer quel modèle a le mieux performé pour la prédiction de l'indice boursier MASI 20.

Explication :

1. **Métriques Quantitatives (MSE, RMSE, MAE)** : Le modèle de **Régression Linéaire** a clairement surpassé le modèle de Régression Forêt Aléatoire. Ses valeurs de MSE, RMSE et MAE sont significativement plus faibles (par exemple, MAE de 11.19 pour la Régression Linéaire contre 83.45 pour la Forêt Aléatoire). Des valeurs plus faibles pour ces métriques indiquent un meilleur ajustement et des prédictions plus précises.
2. **Évaluation Visuelle (Prix Réels vs. Prédits)** : En examinant les graphiques, les prix prédits par le modèle de Régression Linéaire (ligne rouge en pointillés) suivent beaucoup plus fidèlement les prix

réels (ligne bleue). En revanche, les prédictions du modèle de Forêt Aléatoire montrent beaucoup moins de variation et semblent relativement plates par rapport aux fluctuations réelles, ce qui indique qu'il a eu du mal à capturer les tendances et la volatilité.

Conclusion : Sur la base de l'évaluation quantitative et visuelle, le modèle de **Régression Linéaire a démontré une performance supérieure** pour la prédiction de l'indice boursier MASI 20 dans cette configuration unidimensionnelle.